Interview Preparation: Four AI and MCP Projects

7 MAIN SECTIONS • PRINT-READY EDITION

In this guide

1. Travel Planner MCP 1
2. CodeGuard Code Review Assistant 4
3. MCP Studio 7
4. ADK Calendar Assistant 9
5. Interview claims that need careful wording 11
6. Five deeper architecture questions to prepare 12
7. Recommended positioning in the interview 12

I reviewed the source code in all four project ZIP files. The points below are grounded in the implementations, not just the README files.

For each project, this guide includes a **60-second explanation, technical points you can defend, likely interview questions, code references, and limitations**.

The focus is on solution architecture, engineering decisions, AI governance, security, and system integration.

1. Travel Planner MCP

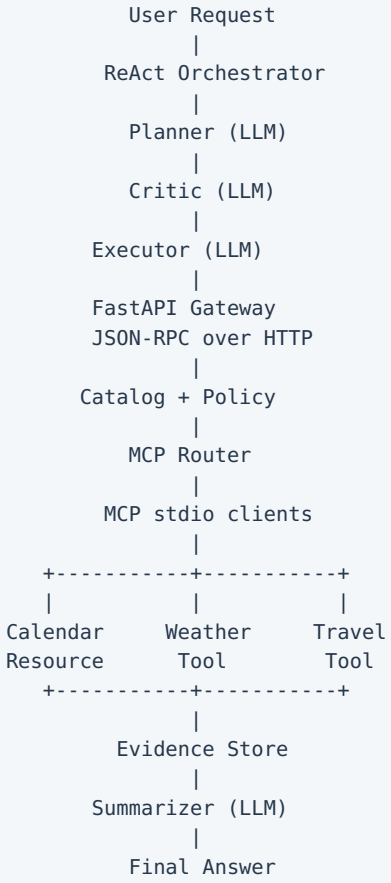
Strongest project for AI agent architecture

60-second interview explanation

> I built a proof-of-concept travel planning agent to explore how an LLM can coordinate multiple services without having direct access to their implementations.
>
> I separated the workflow into four logical roles: Planner, Critic, Executor, and Summarizer. The Planner selects the next action, the Critic reviews it, the Executor prepares tool arguments, and the Summarizer produces an evidence-based response.
>
> The orchestrator communicates with a FastAPI JSON-RPC gateway, which authorizes and routes requests to separate calendar, weather, and travel MCP servers.
>
> I added Pydantic response contracts, tool authorization, bounded execution, timeouts, evidence tracking, and correlated logs.
>
> My main architectural goal was to separate probabilistic AI reasoning from deterministic service execution and authorization.

Architecture to explain

EXAMPLE



Technical points you can justify

Talking point	Code evidence
Explicit ReAct loop and bounded execution	`src/orchestrator/react_loop.py` — `run_react_loop()`, `max_steps=8`
Four role-based LLM interactions	`src/orchestrator/reasoning_adapter.py` — `plan_next_step()`, `review_step()`, `build_tool_args()`, `summarize()`
Structured AI outputs	`src/orchestrator/schemas.py` — `NextStep`, `CriticDecision`, `ToolArgs`, `FinalAnswer`
Catalog authorization	`src/gateway/catalog.py` and `src/gateway/policy.py`
MCP service isolation	`src/gateway/mcp_router.py` — `ClientSession`, `stdio_client`
Timeouts	`src/gateway/mcp_router.py` — `asyncio.wait_for()`
Correlated logging	`src/shared/logging.py`, `src/orchestrator/react_loop.py`
Evidence tracking	`src/orchestrator/memory.py` — `EvidenceStore`

Likely interview questions

Q: Why did you create four roles instead of using a single AI agent?

A: I wanted separation of responsibilities. Planning, reviewing, preparing tool arguments, and summarizing have different requirements. Separate prompts and output contracts make the process easier to inspect and test. However, these are four logical roles using the same reasoning adapter, not four independently deployed agents.

Q: Why introduce a gateway between the LLM and MCP servers?

A: The gateway creates a controlled boundary. It accepts JSON-RPC requests, validates parameters, checks a catalog, identifies the domain, and delegates execution through MCP sessions. The LLM does not directly choose arbitrary server processes.

Q: How did you prevent hallucinated tool calls?

A: I provided the allowed tool catalog to the planner and constrained the expected output using Pydantic models. More importantly, the gateway independently rejects unknown tool names. Model output validation and authorization are separate controls.

Q: How does ReAct work in this project?

A: The loop follows Reason → Act → Observe. The planner selects an action, the system executes it through the gateway, and the result is stored as evidence for the next step. The implementation currently uses a prescribed calendar → weather → travel → finish sequence rather than completely open-ended planning.

Important limitations to acknowledge

- Calendar, weather, and flight responses are deterministic simulated data, not live bookings or forecasts.
- The critic is LLM-based. Gateway authorization is the deterministic control.
- The policy checks a static catalog but does not enforce tenant-specific permissions.
- Evidence is stored in memory for the running workflow.
- The gateway exposes application-specific JSON-RPC methods such as `tools.call`. The router uses the MCP SDK internally.

Best architectural takeaway: An LLM should propose actions, but application-controlled services should validate and execute them.

---

2. CodeGuard Code Review Assistant

Strongest project for AI-assisted engineering governance

60-second interview explanation

> I developed a Python proof-of-concept for AI-assisted code review that combines deterministic metadata extraction with LLM-based risk interpretation.
>
> The system compares Git revisions, parses Python code using Tree-sitter, detects possible hardcoded secrets using regular expressions, and checks Python dependencies against the OSV vulnerability database.
>
> Instead of sending raw source code to the LLM, it generates structured metadata containing complexity metrics, risk indicators, secret findings, and vulnerability information.
>
> The triage layer can use either OpenAI or Claude to generate an assessment. The output is checked against a JSON Schema, with retries for invalid responses, before producing JSON and Markdown reports.
>
> The key design decision was to make the LLM interpret collected evidence rather than rely on it to discover every issue from raw code.

Architecture

EXAMPLE



Technical points you can justify

Talking point	Code evidence
Three-stage review pipeline	`src/run.py` — `run_pipeline()`
Git revision comparison	`extract_structure.py` and `extract_secrets.py` — `get_diff_files()`
Tree-sitter analysis	`src/extractors/extract_structure.py`
Cyclomatic complexity and nesting depth	`calculate_cyclomatic_complexity()`, `calculate_nesting_depth()`
Redacted secret findings	`src/extractors/extract_secrets.py` — `scan_file_for_secrets()`
Dependency vulnerability lookup	`src/extractors/extract_dependencies.py` — `query_osv_api()`
Alternative LLM providers	`src/llm/triage_agent.py`, `triage_agent_openai.py`

Talking point	Code evidence
Validation and retry	`src/validators/validate_schema.py`, `assess_risk()`
Evidence-based reports	`src/reporters/generate_report.py`

Likely interview questions

Q: Why use Tree-sitter instead of asking an LLM to review all the code?

A: Tree-sitter gives reproducible structural information such as function boundaries, parameters, complexity, and nesting depth. That reduces dependence on the LLM for facts that can be extracted programmatically.

Q: How did you reduce the exposure of sensitive source code?

A: The pipeline passes derived metadata to the LLM. The secrets extractor records finding IDs, file locations, secret categories, entropy, and confidence rather than the matched secret values. It is a metadata-minimization design, although file paths and other metadata can still be sensitive.

Q: Why use JSON Schema validation and retries?

A: LLM responses are not always valid application data. The assessment is parsed and validated against a defined schema. If parsing or validation fails, the code retries with error feedback. This checks structural correctness; it does not independently prove every risk claim is factual.

Q: Does the system use OpenAI and Claude together?

A: No. The current CLI selects one provider for a run. The two adapters provide alternative implementations of the same triage stage. There is no cross-model voting or consensus engine.

Q: How are security findings translated into merge recommendations?

A: The LLM generates risk levels, findings, recommendations, and an `allow_merge` result using the extracted metadata and prompt rules. This is advisory output, not a separately enforced CI/CD merge gate.

Important limitations

The implementation is a Python-focused POC. Git diff analysis identifies changed files, but does not precisely analyze only changed lines. Tree-sitter scans Python files in the working tree. The dependency scanner focuses on `requirements.txt` and direct dependencies.

I also identified two code issues that should be corrected before a live demonstration:

- `src/llm/triage_agent_openai.py`, line 49, prints the raw OpenAI API key. This is a security issue. Remove that statement and rotate any key that may have reached logs.
- The Claude adapter uses a schema path that is inconsistent with `SchemaValidator`'s schema directory. This can cause Claude validation to fail.

Best architectural takeaway: Use deterministic tools to collect evidence and AI to help interpret it, with an explicit validation and human-review boundary.

3. MCP Studio

60-second interview explanation

> I built MCP Studio to understand the MCP client-server lifecycle and the behavior of Streamable HTTP.
>
> The first part demonstrates an MCP client establishing a session, initializing the protocol, discovering tools, and invoking tools against a FastAPI-hosted MCP server.
>
> I also implemented ASGI middleware to inspect HTTP headers and request payloads, making protocol behavior easier to troubleshoot.
>
> The second part explores streaming. I implemented shared asynchronous progress-generation logic and exposed it through two mechanisms: MCP progress notifications and a standard REST Server-Sent Events endpoint.
>
> Finally, I containerized the services with Docker Compose and configured Nginx to disable response buffering so progress updates can reach clients incrementally.

Technical points you can justify

Talking point	Code evidence
Stateful MCP HTTP server	`streamable_basics/src/mcp_studio/foundation/server.py`
Session initialization and tool discovery	`streamable_basics/src/mcp_studio/foundation/client.py`
Request inspection	`streamable_basics/src/mcp_studio/debug/request_inspector.py`
Shared async progress generator	`streaming_response/server/src/services/number_stream.py`
MCP progress notifications	`streaming_response/server/src/tools/stream_tool.py`
REST SSE endpoint	`streaming_response/server/src/main.py`
Docker service networking	`streaming_response/docker-compose.yaml`

Talking point	Code evidence
Streaming proxy configuration	`streaming_response/nginx/nginx.conf`

Likely interview questions

Q: What happens during MCP initialization?

A: The client opens the Streamable HTTP transport, creates a `ClientSession`, calls `initialize()`, receives server information, requests `list_tools()`, and then calls a selected tool with typed arguments. The SDK handles the protocol lifecycle.

Q: How is MCP progress different from REST SSE?

A: MCP progress is a protocol-level notification associated with an MCP operation. In this project, the tool uses `ctx.report_progress()`, and the MCP client receives updates through a progress callback.

REST SSE is a conventional HTTP streaming response. The FastAPI endpoint sends text/event-stream frames directly. Both use the same number-progress generator, but they expose it through different protocols.

Q: Why disable Nginx buffering?

A: Buffering can hold response chunks before forwarding them. That defeats the purpose of incremental progress updates. The Nginx configuration disables proxy buffering and caching for the MCP and REST paths and allows longer reads for streaming connections.

Q: What did you learn about ASGI middleware?

A: The inspector wraps the ASGI `receive` callable and observes incoming body chunks without replacing the MCP SDK's protocol handling. That helps separate transport-level troubleshooting from MCP application logic.

Important limitations

- The MCP clients are command-line applications; the **servers** are FastAPI-hosted.
- The streaming example generates numbered progress events, not LLM-generated token streams.
- The request inspector prints raw headers and payloads. It needs redaction and controlled logging before production use.
- The code demonstrates protocol behavior, not a complete production MCP security platform.

Best architectural takeaway: Protocol design, session management, streaming behavior, and reverse-proxy configuration must work together for reliable distributed integrations.

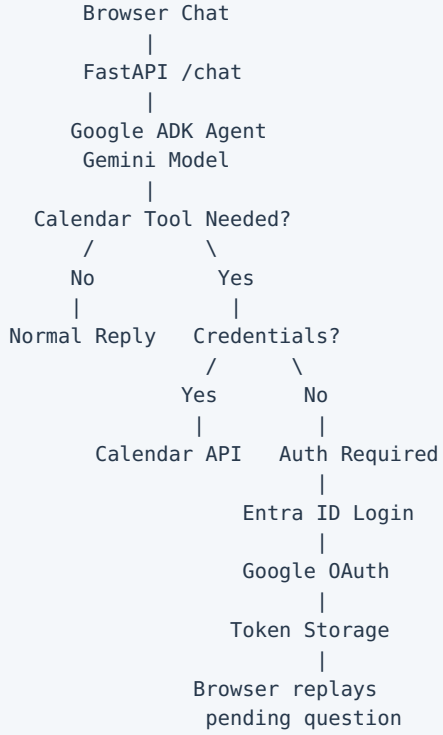
4. ADK Calendar Assistant

60-second interview explanation

> I built a read-only calendar assistant using Google's Agent Development Kit, Gemini, FastAPI, and the Google Calendar API.
>
> The agent supports conversational requests to retrieve upcoming events and check events on a requested date.
>
> The interesting part was the identity integration. I implemented a two-stage authentication process: Microsoft Entra ID for user sign-in, followed by delegated Google OAuth to grant read-only calendar access.
>
> I used an authentication sentinel so the agent only requests authentication when a calendar tool actually needs credentials.
>
> I also added token refresh handling, ADK in-memory conversation sessions, and a browser chat flow that saves a pending question during authentication and replays it after the user returns.

Architecture

EXAMPLE



Technical points you can justify

Talking point	Code evidence
Gemini-powered ADK agent	`agent/calendar_agent.py` — `create_calendar_agent()`
Read-only Google Calendar operations	`agent/calendar_tools.py`
Entra ID OIDC authentication	`auth/entra_oidc.py`
Delegated Google OAuth with PKCE	`auth/google_oauth.py`
Token refresh	`agent/calendar_tools.py` — `_service()`
ADK session reuse	`agent/runner.py` — `InMemorySessionService`
Authentication callback handling	`server/app.py`
Pending-question replay	`server/static/chat.html`

Likely interview questions

Q: Why use two authentication providers?

A: Entra ID establishes the application's signed-in user identity. Google OAuth separately grants permission to access that user's Google Calendar. This separates application identity from delegated access to an external resource.

Q: Why request read-only calendar permissions?

A: The assistant only needs to read events. The code requests the `calendar.readonly` scope and exposes no create, update, or delete tools. That limits the authority granted to the assistant.

Q: How does token refresh work?

A: Before constructing the Calendar API service, the code checks whether the stored credentials have expired. When a refresh token is available, it invokes the Google credentials refresh method.

Q: How does the assistant continue after authentication?

A: When a calendar tool needs authorization, it returns an authentication sentinel. The runner translates that into an `auth_required` response. The browser saves the question in `sessionStorage`, starts authentication, and resubmits the question after the OAuth redirects complete.

This is **question replay**, not restoration of an interrupted ADK tool execution.

Important limitations

The date-availability implementation filters up to 50 upcoming events by date. It does not use Google's FreeBusy API or query the full requested day's time range. Therefore, it should be described as a basic date-based event check, not an authoritative availability service.

The in-memory session and token stores are suitable for a demonstration. Before production use, the project would need persistent storage, stronger session isolation, and explicit validation that the Google OAuth callback is linked to the correct signed-in Entra user.

Best architectural takeaway: Agent tool access should follow authenticated identity, delegated authorization, and least-privilege design.

5. Interview claims that need careful wording

These are the most important distinctions I found in the actual code.

Project	Avoid saying	Better wording
Travel Planner	Fully autonomous multi-agent system	Role-separated ReAct workflow with a prescribed execution sequence
Travel Planner	Production-grade tenant security	Catalog authorization prototype; tenant-specific controls remain future work
CodeGuard	Complete automated secure code review	AI-assisted metadata-based risk assessment POC
CodeGuard	Both LLMs jointly assess risks	Selectable OpenAI or Claude triage adapters
MCP Studio	Production-ready MCP platform	MCP protocol and streaming integration experiments
ADK Calendar	Full calendar availability engine	Upcoming-event retrieval and basic date-based availability checks
ADK Calendar	Agent execution resumes after OAuth	Browser replays the pending question after sign-in

6. Five deeper architecture questions to prepare

These are especially relevant to a senior solutions architecture interview.

1. **Where do you enforce authorization when an AI agent calls tools?**
   Explain why a prompt is not sufficient. Use Travel Planner's gateway policy and ADK's delegated OAuth as two different authorization boundaries.
2. **How do you validate AI-generated decisions?**
   Contrast Travel Planner's Pydantic output models with CodeGuard's JSON Schema validation and retry loop. Explain why schema-valid does not mean factually correct.
3. **How do you handle failures in distributed AI workflows?**
   Use Travel Planner's maximum-step limit, MCP call timeouts, JSON-RPC error handling, and correlated logs. Acknowledge that retries, circuit breakers, and recovery are not fully implemented.
4. **How do you protect sensitive enterprise information when using LLMs?**
   Describe CodeGuard's metadata-minimization approach, ADK's least-privilege Google scope, and the risks of logging credentials or raw request bodies.
5. **How would you evolve these prototypes into an enterprise platform?**
   Discuss centralized authorization, per-tenant isolation, audit logging, persistent state, deterministic evaluation, human approval for consequential actions, observability, automated security tests, and controlled deployment.

7. Recommended positioning in the interview

Rather than presenting these as four unrelated applications, use them to demonstrate four related areas of architectural learning.

Project	Primary skill to demonstrate
Travel Planner MCP	AI orchestration, separation of duties, service integration, and agent guardrails
CodeGuard	AI-assisted engineering governance, evidence extraction, and output validation
MCP Studio	Protocol-level integration, Streamable HTTP, and asynchronous communication
ADK Calendar Assistant	Agent development, enterprise identity integration, and delegated authorization

A strong final connection across the projects is:

> These projects helped me explore different parts of responsible enterprise AI integration. Travel Planner focused on separating AI decisions from controlled execution. CodeGuard focused on using deterministic evidence to support AI-assisted decisions. MCP Studio helped me understand communication protocols and streaming behavior. The Calendar Assistant focused on identity, delegated access, and conversational state.
>
> Across all four, the recurring architectural principle was to keep AI reasoning separate from authentication, authorization, validation, and service execution.

Verification scope: The repository code was inspected and Python syntax was checked. The complete applications were not executed against live LLM providers, OAuth providers, or external services. The interview explanations distinguish what is implemented from what still requires production hardening.